When using a container topology, you need a machine with a container runtime. With Kubernetes, however, you switch to an orchestration topology, meaning that you need a cluster. A Kubernetes *cluster* is a set of worker machines (*nodes*) that run containerized applications. Every cluster has at least one worker node. With minikube. you can easily create a single-node cluster on your local machine. In production, you'll use a cluster managed by a cloud provider.

A Kubernetes cluster comprises machines called *worker nodes* on which your containerized applications are deployed. They provide capacity such as CPU, memory, network, and storage so that the containers can run and connect to a network.

The *control plane* is the container orchestration layer that manages the worker nodes. It exposes the API and interfaces to define, deploy, and manage the life cycle of containers. It offers all the essential elements that implement the typical features of an orchestrator, like cluster management, scheduling, and health monitoring.

NOTE In the context of container orchestration, *scheduling* means matching a container instance with a node where it will be run. The matching is based on a set of criteria, including the availability of enough computational resources to run the container on the node.

You can interact with Kubernetes through a CLI client, kubectl, which communicates with the control plane to perform some operations on the worker nodes. A client doesn't interact with the worker nodes directly. Figure 2.10 shows the high-level components of the Kubernetes architecture.